

Arrays and Strings

The foundation of every interview. Master this chapter first.

This chapter covers the most commonly asked topic in product company interviews. Every company, from Zomato to PayPal, asks array and string problems. You will learn what arrays really are, how strings work in Python, the most important patterns, and solve real interview problems step by step.

What you will learn	Why it matters
What an array is in memory	Every data structure builds on this
How strings work in Python	60% of easy problems involve strings
The two pointer pattern	Most common pattern in array interviews
How to go from brute force to optimal	What interviewers actually want to see
Pattern recognition: spot the trick	The real skill interviews test

1.1

What is an array really?

You already know how to create a list in Python. But what is actually happening in memory when you do that? Understanding this will help you make better decisions in every problem you solve.

Imagine a row of boxes in a post office. Each box has a number (its address). When you want box number 5, you go directly to box 5. You do not check box 1, then box 2, then box 3. You jump straight there. That is why accessing any index in a list is $O(1)$.

Python — creating and accessing arrays

```
# Creating a list
fruits = ['mango', 'banana', 'apple', 'grapes']
# index:      0          1          2          3

# Accessing by index —  $O(1)$ , always instant
print(fruits[0])    # mango
print(fruits[2])    # apple
print(fruits[-1])   # grapes (negative index = from end)
print(fruits[-2])   # apple

# Length of list —  $O(1)$ 
print(len(fruits))  # 4

# Modifying by index —  $O(1)$ 
fruits[1] = 'papaya'
print(fruits)  # ['mango', 'papaya', 'apple', 'grapes']
```

1.2

Important array operations and their complexity

Before solving problems, you must know the cost of each operation. This affects every solution you write.

Operation	Python code	Complexity	Why
Access by index	<code>arr[i]</code>	$O(1)$	Direct jump to position
Append at end	<code>arr.append(x)</code>	$O(1)^*$	Adds at end, no shifting
Insert at start	<code>arr.insert(0,x)</code>	$O(n)$	Shifts everything right
Delete from start	<code>arr.pop(0)</code>	$O(n)$	Shifts everything left
Delete from end	<code>arr.pop()</code>	$O(1)$	No shifting needed
Search for value	<code>x in arr</code>	$O(n)$	Checks every element
Slice	<code>arr[a:b]</code>	$O(b-a)$	Creates a copy
Sort	<code>arr.sort()</code>	$O(n \log n)$	Python's Timsort
Length	<code>len(arr)</code>	$O(1)$	Stored internally

The one you will forget and it will cost you

`arr.insert(0, x)` and `arr.pop(0)` are both $O(n)$. Every existing element shifts by one position. If you call these inside a loop, your solution becomes $O(n^2)$ without realising it. Use `collections.deque` if you need fast inserts and deletes at both ends.

1.3

Strings in Python

A string in Python is like a read-only list of characters. Everything you know about array indexing applies to strings. The critical difference: strings are immutable. You cannot change a character in place. You must create a new string.

Python — strings behave like arrays

```
name = 'interview'

# index: 0123456789

# Access by index — 0(1)

print(name[0])      # i
print(name[-1])     # w
print(name[2:5])    # ter (slicing, creates new string)

# Strings are immutable

# name[0] = 'I'      <- This causes an error!

# To 'modify' a string, create a new one
new_name = 'I' + name[1:] # 'Interview'

# Useful string methods

s = ' Hello World '

print(s.lower())      # ' hello world '
print(s.upper())      # ' HELLO WORLD '
print(s.strip())      # 'Hello World' (removes spaces)
print(s.split())      # ['Hello', 'World']
print('hello'.replace('l','r')) # 'herro'

# Check characters

print('abc123'.isalpha()) # False (has numbers)
print('abc'.isalpha())    # True
print('123'.isdigit())    # True
```

String concatenation inside a loop is $O(n^2)$

Every time you do `result = result + char`, Python creates a brand new string. Do this n times and you have n string creations = $O(n^2)$ time.

The fix: collect characters in a list, then join at the end.

WRONG: `result = ""` then `result += char` (inside loop)

RIGHT: `chars = []` then `chars.append(char)` then `"".join(chars)`

1.4

The two pointer pattern

The two pointer pattern is the most important pattern in this chapter. It appears in at least 30 percent of array and string problems in interviews. Learn to spot it and you automatically know how to approach those problems.

The idea: instead of using nested loops (which gives $O(n^2)$), you use two variables that point to positions in the array and move them toward each other or in the same direction. This gives $O(n)$.

When to use two pointers

The array or string is sorted, OR you are looking for a pair that satisfies some condition, OR you need to compare elements from both ends, OR the problem mentions 'in-place'.

Two pointer — the basic template

```
def two_pointer_template(arr):  
    left = 0          # starts at beginning  
    right = len(arr) - 1 # starts at end  
  
    while left < right:  
        # do something with arr[left] and arr[right]  
  
        if some_condition:  
            left += 1      # move left pointer right  
        else:  
            right -= 1     # move right pointer left  
  
    # left and right have crossed — we are done
```

1.5

Solved examples

Read each solved example carefully. For every example: first read the problem and try to think of a solution yourself. Then read the brute force. Then read the optimized solution. Then re-read the interviewer note. That note is the most important part.

Solved Example 1.1

[Easy]

Find the maximum element in an array

Problem: Given a list of integers, find and return the largest number. Example: [3, 1, 7, 2, 9, 4]
-> return 9.

Step 1: Brute force solution (always start here)

Brute force — Python

```
# Brute force: compare every element with every other element
def find_max_brute(nums):
    n = len(nums)
    for i in range(n):
        is_max = True
        for j in range(n):          # nested loop
            if nums[j] > nums[i]:
                is_max = False
                break
        if is_max:
            return nums[i]

# Time: O(n²)  Space: O(1)
# Problem: checks every pair — very slow
```

Brute force complexity: Time: $O(n^2)$, Space: $O(1)$

Step 2: Optimized solution

Optimized — Python

```
# Optimized: one pass, track the max seen so far
def find_max(nums):
    max_val = nums[0]      # assume first is max

    for num in nums:      # visit each element once
        if num > max_val:
            max_val = num  # update if we find bigger

    return max_val

# Test it
print(find_max([3, 1, 7, 2, 9, 4]))  # 9
print(find_max([-5, -1, -8, -2]))    # -1

# Time: O(n)  Space: O(1)
```

Optimized complexity: Time: $O(n)$, Space: $O(1)$

What the interviewer is checking

This tests whether you think about the problem before coding. The brute force compares every pair which is $O(n^2)$. The insight: you only need one pass. Keep track of the best answer so far. This 'running best' pattern appears in dozens of problems.

Common mistake to avoid

Do not initialise `max_val = 0`. If all numbers are negative, you will return 0 which is wrong. Always initialise to `nums[0]`.

Solved Example 1.2

[Easy]

Reverse an array in place

Problem: Given a list of integers, reverse it in place without creating a new list. Example: [1, 2, 3, 4, 5] -> [5, 4, 3, 2, 1]. Do not use `arr.reverse()` or `arr[::-1]`.

Step 1: Brute force solution (always start here)

Brute force — Python

```
# Brute force: create a new list in reverse order
def reverse_brute(arr):
    result = []
    for i in range(len(arr)-1, -1, -1): # loop backwards
        result.append(arr[i])
    return result

# Time: O(n)  Space: O(n)  <- extra list created
# Problem: uses O(n) extra space. 'In place' means O(1) space.
```

Brute force complexity: Time: $O(n)$, Space: $O(n)$

Step 2: Optimized solution

Optimized — Python

```
# Optimized: two pointers swap from both ends
def reverse_inplace(arr):
    left = 0
    right = len(arr) - 1

    while left < right:
        # swap the two elements
        arr[left], arr[right] = arr[right], arr[left]

        left += 1 # move inward
        right -= 1 # move inward

    return arr

# Test it
print(reverse_inplace([1,2,3,4,5])) # [5,4,3,2,1]
print(reverse_inplace([1,2,3,4]))   # [4,3,2,1]

# Time: O(n) Space: O(1) <- no extra memory
```

Optimized complexity: Time: $O(n)$, Space: $O(1)$

What the interviewer is checking

This is the classic two pointer introduction. The interviewer wants to see that you know how to swap in Python ($a, b = b, a$) and that you understand in-place means $O(1)$ space. Many candidates create a new list and lose marks.

Common mistake to avoid

The swap in Python is clean: `arr[left], arr[right] = arr[right], arr[left]`. No need for a temp variable like in other languages.

Solved Example 1.3

[Easy]

Check if a string is a palindrome

Problem: Given a string, return True if it reads the same forward and backward. Example: 'racecar' -> True, 'madam' -> True, 'hello' -> False. Ignore case (treat 'A' same as 'a').

Step 1: Brute force solution (always start here)

Brute force — Python

```
# Brute force: reverse the string and compare
def is_palindrome_brute(s):
    s = s.lower()
    reversed_s = s[::-1]      # creates a new reversed string
    return s == reversed_s

# Time: O(n)  Space: O(n)  <- reversed string uses extra memory
```

Brute force complexity: Time: $O(n)$, Space: $O(n)$

Step 2: Optimized solution

Optimized — Python

```
# Optimized: two pointers from both ends
def is_palindrome(s):
    s = s.lower()          # handle case
    left = 0
    right = len(s) - 1

    while left < right:
        if s[left] != s[right]:
            return False   # mismatch found
        left += 1
        right -= 1

    return True            # no mismatch found

# Test it
print(is_palindrome('racecar'))    # True
print(is_palindrome('Madam'))      # True
print(is_palindrome('hello'))      # False

# Time: O(n)  Space: O(1)
```

Optimized complexity: Time: $O(n)$, Space: $O(1)$

What the interviewer is checking

Palindrome problems always signal two pointers. The interviewer is checking if you see that you can compare from both ends simultaneously instead of reversing the entire string. Also checks if you handle edge cases like case sensitivity and single character strings.

Common mistake to avoid

Single character strings are always palindromes. The while loop handles this correctly — `left < right` is `False` immediately so it returns `True`. Always trace through edge cases before submitting.

Solved Example 1.4

[Easy]

Find two numbers that add up to a target

Problem: Given a list of integers and a target number, return the indices of two numbers that add up to the target. Example: `nums=[2,7,11,15]`, `target=9` -> return `[0,1]` because `nums[0]+nums[1]=9`. Assume exactly one solution exists.

Step 1: Brute force solution (always start here)

Brute force — Python

```
# Brute force: check every pair

def two_sum_brute(nums, target):
    n = len(nums)
    for i in range(n):          # first number
        for j in range(i+1, n): # second number
            if nums[i] + nums[j] == target:
                return [i, j]
    return []

# Time: O(n^2)  Space: O(1)
# Checks every possible pair – very slow for large inputs
```

Brute force complexity: Time: $O(n^2)$, Space: $O(1)$

Step 2: Optimized solution

Optimized — Python

```
# Optimized: hashmap stores numbers we have already seen
def two_sum(nums, target):
    seen = {}    # stores {number: index}

    for i, num in enumerate(nums):
        complement = target - num    # what do we need?

        if complement in seen:       # O(1) lookup
            return [seen[complement], i]

        seen[num] = i    # store current number

    return []

# Test it
print(two_sum([2,7,11,15], 9))    # [0, 1]
print(two_sum([3,2,4], 6))        # [1, 2]

# Time: O(n)  Space: O(n)  <- hashmap uses extra space
```

Optimized complexity: Time: $O(n)$, Space: $O(n)$

What the interviewer is checking

Two Sum is the most famous interview problem. It tests whether you know to use a hashmap to trade space for time. The key insight: instead of searching for the complement ($O(n)$ each time), store what you have seen in a dict and look it up in $O(1)$. This pattern — use a hashmap to avoid a nested loop — appears in at least 20 percent of hashmap problems.

Common mistake to avoid

A common mistake: checking 'if num in seen' before storing. This misses the case where the same element is used twice. Always check first, then store.

Solved Example 1.5

[Easy]

Remove duplicates from a sorted array

Problem: Given a sorted array, remove duplicates in place so each element appears only once. Return the new length. Do not allocate extra space. Example: [1,1,2,3,3,4] -> modify to [1,2,3,4,...] and return 4.

Step 1: Brute force solution (always start here)

Brute force — Python

```
# Brute force: use a set to track unique elements
def remove_duplicates_brute(nums):
    unique = list(set(nums))    # creates new list
    unique.sort()              # set does not preserve order
    for i in range(len(unique)):
        nums[i] = unique[i]
    return len(unique)

# Time: O(n log n)  Space: O(n)  <- set uses extra memory
```

Brute force complexity: Time: $O(n \log n)$, Space: $O(n)$

Step 2: Optimized solution

Optimized — Python

```
# Optimized: two pointers – slow and fast
def remove_duplicates(nums):
    if not nums:
        return 0

    slow = 0 # points to last unique element placed

    for fast in range(1, len(nums)): # fast scans ahead
        if nums[fast] != nums[slow]: # found new unique
            slow += 1
            nums[slow] = nums[fast] # place it after slow

    return slow + 1 # length of unique section

# Test it
arr = [1,1,2,3,3,4]
length = remove_duplicates(arr)
print(length) # 4
print(arr[:length]) # [1, 2, 3, 4]

# Time: O(n) Space: O(1) <- no extra memory
```

Optimized complexity: Time: $O(n)$, Space: $O(1)$

What the interviewer is checking

This introduces the slow-fast pointer variant. Slow pointer tracks where to write next. Fast pointer scans for new unique elements. The interviewer is checking if you can work in-place without extra memory. The array being sorted is the key — duplicates are always adjacent.

Common mistake to avoid

This only works because the array is sorted. If the array were unsorted, you would need a different approach. Always check: is the input sorted? That information changes the optimal solution.

Solved Example 1.6

[Medium]

Find the largest sum of a contiguous subarray

Problem: Given an array of integers (can include negative numbers), find the contiguous subarray with the largest sum and return that sum. Example: $[-2, 1, -3, 4, -1, 2, 1, -5, 4] \rightarrow 6$ (subarray $[4, -1, 2, 1]$).

Step 1: Brute force solution (always start here)

Brute force — Python

```
# Brute force: try every possible subarray
def max_subarray_brute(nums):
    n = len(nums)
    max_sum = float('-inf') # negative infinity

    for i in range(n):      # start of subarray
        for j in range(i, n): # end of subarray
            current = sum(nums[i:j+1])
            max_sum = max(max_sum, current)

    return max_sum

# Time:  $O(n^2)$  Space:  $O(1)$ 
```

Brute force complexity: Time: $O(n^2)$, Space: $O(1)$

Step 2: Optimized solution

Optimized — Python

```
# Optimized: Kadane's algorithm – one pass

def max_subarray(nums):
    max_sum = nums[0] # best sum found so far
    current_sum = nums[0] # sum of current subarray

    for num in nums[1:]: # start from second element
        # Decision: extend current subarray OR start fresh
        current_sum = max(num, current_sum + num)
        max_sum = max(max_sum, current_sum)

    return max_sum

# Test it
print(max_subarray([-2,1,-3,4,-1,2,1,-5,4])) # 6
print(max_subarray([1])) # 1
print(max_subarray([-1,-2,-3])) # -1

# Time: O(n) Space: O(1)
```

Optimized complexity: Time: $O(n)$, Space: $O(1)$

What the interviewer is checking

Kadane's algorithm is one of the most elegant solutions in DSA. The key decision at each step: is it better to extend the current subarray or start fresh from this element? If $\text{current_sum} + \text{num} < \text{num}$, then current_sum was negative and we should start over. This is a medium difficulty problem but appears very frequently. Interviewers use it to test if you can identify the optimal substructure.

Common mistake to avoid

Initialise both variables to `nums[0]`, not 0. If all numbers are negative, returning 0 is wrong. The answer is the least negative number.

Pattern recognition: how to spot two pointers

This is the section most books skip. They teach you the algorithm but not how to recognize when to use it. Here are 6 problem statements. Each one is a disguised two-pointer problem. Read each one and notice what signals reveal the pattern.

Problem A: Given a sorted array and a target, find if any two elements add up to target.

Signal and approach

Sorted array + find a pair = two pointers from both ends. Move left right if sum too small, move right left if sum too big.

Problem B: Given a string, check if it is a palindrome ignoring spaces and non-letter characters.

Signal and approach

Compare characters from both ends = two pointers. Skip non-letters by moving the pointer until you find a letter.

Problem C: Given an array of 0s and 1s, move all 0s to the left and 1s to the right in place.

Signal and approach

In-place rearrangement with two categories = two pointers. Left pointer collects 0s, right pointer collects 1s.

Problem D: Given a sorted array, return a new array of squares of each element, also sorted.

Signal and approach

Sorted array + squares can cause order issues (negatives squared become large) = two pointers from both ends. Largest square is always at one of the two ends.

Problem E: Given a string, find the length of the longest substring without repeating characters.

Signal and approach

This is actually sliding window (Chapter 3), not two pointers. Signals: 'longest substring' + 'without repeating'. Do not confuse with two pointers.

Problem F: Given an array, remove all occurrences of a specific value in place and return new length.

Signal and approach

In-place removal = slow-fast pointer variant (same as remove duplicates). Slow tracks where to write, fast scans for valid elements.

The signals that say 'two pointers'

1. The array or string is sorted AND you need to find a pair.
2. The problem says 'in place' (meaning $O(1)$ space).
3. You need to compare elements from opposite ends.
4. You need to remove/move elements while scanning.
5. The problem involves subarrays starting and ending at different positions.

Practice problems

Attempt every problem yourself before reading the hint. Spend at least 20 minutes on each problem. If you are stuck after 20 minutes, read only the hint — not a solution. Code it yourself after the hint.

Your target times

Easy problems: solve with working code in 15 to 20 minutes.

Medium problems: solve with working code in 30 to 40 minutes.

If you take longer: that is fine right now. Speed comes with practice. Accuracy first, then speed.

Practice Problem 1.1

[Easy]

Count negative numbers in a sorted matrix

Given a 2D matrix where each row is sorted in descending order, count the total number of negative numbers. Example: `[[4,3,2,-1],[3,2,1,-1],[1,1,-1,-2],[-1,-1,-2,-3]]` -> 8.

Hint (read only after 20 minutes of trying)

The matrix rows are sorted. For each row, you can find where negatives start using a pointer starting from the right. Move left while the number is negative. You do not need a nested loop searching every element.

Practice Problem 1.2

[Easy]

Check if two strings are anagrams

Given two strings `s` and `t`, return `True` if `t` is an anagram of `s`. An anagram uses the same characters in different order. Example: `'anagram'` and `'nagaram'` -> `True`. `'rat'` and `'car'` -> `False`.

Hint (read only after 20 minutes of trying)

Count the frequency of each character in both strings. If frequencies match, they are anagrams. Think about which Python tool counts frequencies in one line.

Practice Problem 1.3

[Easy]

Move all zeros to the end

Given an array, move all zeros to the end while maintaining the relative order of non-zero elements. Do it in place. Example: `[0,1,0,3,12] -> [1,3,12,0,0]`.

Hint (read only after 20 minutes of trying)

Use the slow-fast pointer approach from Solved Example 1.5. Slow pointer tracks where to place the next non-zero element. Fast pointer scans for non-zero elements.

Practice Problem 1.4

[Easy]

Find the single number

Given an array where every element appears exactly twice except one element which appears once, find and return that single element. Example: `[4,1,2,1,2] -> 4`. Solve in $O(n)$ time and $O(1)$ space.

Hint (read only after 20 minutes of trying)

A hashmap solution is $O(n)$ time but $O(n)$ space. For $O(1)$ space, think about the XOR operation. Any number XOR itself equals 0. Any number XOR 0 equals itself. What happens if you XOR all elements together?

Practice Problem 1.5

[Easy]

Reverse words in a sentence

Given a string sentence, reverse the order of words. Example: 'the sky is blue' -> 'blue is sky the'.
Remove extra spaces if present.

Hint (read only after 20 minutes of trying)

Python's `split()` method handles multiple spaces and gives you a clean list of words. Then reverse the list and join back with spaces. Think about what built-in tools Python gives you.

Practice Problem 1.6

[Easy]

Best time to buy and sell stock

Given an array where `prices[i]` is the price of a stock on day *i*, find the maximum profit you can make by buying on one day and selling on a later day. You must buy before you sell. Return 0 if no profit is possible. Example: `[7,1,5,3,6,4]` -> 5 (buy at 1, sell at 6).

Hint (read only after 20 minutes of trying)

Track the minimum price seen so far as you scan left to right. At each day, calculate `profit = current price minus minimum so far`. Track the maximum profit seen. This is the 'running best' pattern from Solved 1.1.

Practice Problem 1.7

[Easy]

Find majority element

Given an array of size *n*, find the element that appears more than $n/2$ times. It is guaranteed that the majority element always exists. Example: `[3,2,3]` -> 3. `[2,2,1,1,1,2,2]` -> 2.

Hint (read only after 20 minutes of trying)

A hashmap counting frequencies is the obvious $O(n)$ space solution. For $O(1)$ space, look up Boyer-Moore Voting Algorithm. The idea: maintain a candidate and a count. When count drops to 0, change candidate.

Practice Problem 1.8

[Easy]

Merge two sorted arrays

Given two sorted arrays `nums1` and `nums2`, merge `nums2` into `nums1` so the result is sorted. `nums1` has extra space at the end for the elements of `nums2`. Example: `nums1=[1,2,3,0,0,0]` `m=3`, `nums2=[2,5,6]` `n=3` -> `[1,2,2,3,5,6]`.

Hint (read only after 20 minutes of trying)

Start filling from the END of `nums1` not the beginning. Use three pointers: one at end of real `nums1`, one at end of `nums2`, one at end of total space. This avoids shifting elements and keeps it $O(n)$.

Practice Problem 1.9

[Medium]

Container with most water

Given an array `height` where `height[i]` is the height of a vertical line at position `i`, find two lines that together with the x-axis form a container that holds the most water. Example: `[1,8,6,2,5,4,8,3,7]` -> 49.

Hint (read only after 20 minutes of trying)

Two pointers from both ends. The area is $\min(\text{height}[\text{left}], \text{height}[\text{right}]) * (\text{right} - \text{left})$. Move the pointer pointing to the shorter line inward. Why? Moving the taller line inward can only decrease or maintain the height limit while also decreasing the width. Moving the shorter line gives a chance to find taller.

Practice Problem 1.10

[Medium]

Three sum

Given an array, find all unique triplets that add up to zero. Example: `[-1,0,1,2,-1,-4]` -> `[[-1,-1,2],[-1,0,1]]`. No duplicate triplets in the answer.

Hint (read only after 20 minutes of trying)

Sort the array first. Fix one element with an outer loop. For the remaining two elements, use two pointers (this is two sum on a sorted array). Skip duplicates carefully to avoid duplicate triplets in the answer.

Solutions to practice problems

Before reading solutions

Only read a solution after you have a working attempt, even if it is slow. If you look at solutions without trying, you are building the illusion of skill not the skill itself.

Solution 1.1: Count negatives in sorted matrix

Solution 1.1

```
def count_negatives(grid):
    count = 0
    for row in grid:
        # start from right, move left while negative
        right = len(row) - 1
        while right >= 0 and row[right] < 0:
            count += 1
            right -= 1
    return count

# Time: O(m + n)   Space: O(1)
# m = rows, n = columns
```

Solution 1.2: Check anagrams

Solution 1.2

```
from collections import Counter

def is_anagram(s, t):
    if len(s) != len(t):
        return False
    return Counter(s) == Counter(t)

# Time: O(n) Space: O(1)
# O(1) space because only 26 lowercase letters possible
```

Solution 1.3: Move zeros to end

Solution 1.3

```
def move_zeros(nums):
    slow = 0
    for fast in range(len(nums)):
        if nums[fast] != 0:
            nums[slow] = nums[fast]
            slow += 1
    # fill rest with zeros
    while slow < len(nums):
        nums[slow] = 0
        slow += 1

# Time: O(n) Space: O(1)
```

Solution 1.4: Single number

Solution 1.4

```
def single_number(nums):  
    result = 0  
    for num in nums:  
        result ^= num    # XOR  
    return result  
  
# XOR trick: a ^ a = 0, a ^ 0 = a  
# All pairs cancel out, leaving the single number  
# Time: O(n)  Space: O(1)
```

Solution 1.5: Reverse words

Solution 1.5

```
def reverse_words(s):  
    words = s.split()    # handles extra spaces  
    words.reverse()      # in place  
    return ' '.join(words)  
  
# Time: O(n)  Space: O(n)
```

Solution 1.6: Best time to buy and sell stock

Solution 1.6

```
def max_profit(prices):  
    min_price = float('inf')  
    max_profit = 0  
    for price in prices:  
        min_price = min(min_price, price)  
        max_profit = max(max_profit, price - min_price)  
    return max_profit  
  
# Time: O(n)  Space: O(1)
```

Solution 1.7: Majority element

Solution 1.7

```
def majority_element(nums):  
    candidate = None  
    count = 0  
    for num in nums:  
        if count == 0:  
            candidate = num  
        count += (1 if num == candidate else -1)  
    return candidate  
  
# Boyer-Moore Voting Algorithm  
# Time: O(n) Space: O(1)
```

Solution 1.8: Merge sorted arrays

Solution 1.8

```
def merge(nums1, m, nums2, n):  
    p1 = m - 1      # end of real nums1  
    p2 = n - 1      # end of nums2  
    p = m + n - 1   # end of total space  
  
    while p1 >= 0 and p2 >= 0:  
        if nums1[p1] > nums2[p2]:  
            nums1[p] = nums1[p1]  
            p1 -= 1  
        else:  
            nums1[p] = nums2[p2]  
            p2 -= 1  
        p -= 1  
  
    # remaining elements of nums2  
    nums1[:p2+1] = nums2[:p2+1]  
  
    # Time: O(m+n)  Space: O(1)
```

Solution 1.9: Container with most water

Solution 1.9

```
def max_water(height):
    left, right = 0, len(height) - 1
    max_area = 0
    while left < right:
        h = min(height[left], height[right])
        area = h * (right - left)
        max_area = max(max_area, area)
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1
    return max_area

# Time: O(n)  Space: O(1)
```

Solution 1.10: Three sum

Solution 1.10

```
def three_sum(nums):
    nums.sort()
    result = []
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i-1]:
            continue # skip duplicate for first element
        left = i + 1
        right = len(nums) - 1
        while left < right:
            s = nums[i] + nums[left] + nums[right]
            if s == 0:
                result.append([nums[i], nums[left], nums[right]])
                while left < right and nums[left] == nums[left+1]:
                    left += 1 # skip duplicates
                while left < right and nums[right] == nums[right-1]:
                    right -= 1 # skip duplicates
                left += 1
                right -= 1
            elif s < 0:
                left += 1
            else:
                right -= 1
    return result

# Time: O(n^2) Space: O(1) excluding output
```

Chapter 1 Summary

Pattern	When to use it	Complexity improvement
Running best (min/max so far)	Find max, min, best profit in one pass	$O(n^2) \rightarrow O(n)$
Two pointers from ends	Sorted array, pair sum, palindrome check	$O(n^2) \rightarrow O(n)$
Slow-fast pointers	In-place removal, duplicates, zeros	$O(n^2) \rightarrow O(n)$
Hashmap for complement	Two sum, frequency count, pair finding	$O(n^2) \rightarrow O(n)$
Kadane's algorithm	Maximum subarray sum	$O(n^2) \rightarrow O(n)$

What to do before Chapter 2

Solve all 10 practice problems without looking at solutions first. Then solve these 5 problems on LeetCode to verify you can handle slight variations: #26 Remove Duplicates, #121 Best Time to Buy Stock, #53 Maximum Subarray, #1 Two Sum, #125 Valid Palindrome. If you can solve these without help, you are ready for Chapter 2.

End of Chapter 1 — Next: Chapter 2: Hashmaps and Sets